

Maximum Average Subarray of Length $\geq L$

长度 $\geq L$ 的最大平均子数组 —— 一次完整的推理轨迹

From wrong intuition to a binary-search feasibility check · 对应 LeetCode 644 (Maximum Average Subarray II)

这份笔记不只给“干净解法”。它把真实推演里每一次错误直觉是怎么冒出来的、卡在哪、被什么反例或 ground truth 打掉、又怎么修正原样记录下来。最值钱的不是终解，而是这条试错链 (the reasoning trace) —— 它就是 Staff 级面试现场真正发生的事情。

0. 问题 / Problem

给定整型数组 $a[0..n-1]$ 和整数 L 。求一个长度 $\geq L$ 的连续子数组，使其平均值最大，返回该最大平均值。

Given an integer array and an integer L , find a contiguous subarray of length at least L whose average is maximum; return that average.

约束： $1 \leq L \leq n \leq 1e5$ ， $-1e4 \leq a[i] \leq 1e4$ （有负数），答案误差 $1e-5$ 即可。要求复杂度优于 $O(n^2)$ 。

记前缀和 $P[k] = a[0] + \dots + a[k-1]$ ， $P[0] = 0$ 。任一区间 $(j, i]$ 的平均值为：

$$\text{avg}(j, i] = (P[i] - P[j]) / (i - j), \text{ 约束 } i - j \geq L$$

为什么“长度 $\geq L$ ”比“定长 L ”难：定长时窗口唯一、滑窗 $O(n)$ 即可；而“ $\geq L$ ”意味着对每个右端点 i ，左端点 j 可在一整段区间里自由选，候选区间有 $O(n^2)$ 个。

1. 试错轨迹 / The Reasoning Trace

Attempt 1 — 扩窗剪枝（双指针的诱惑）

直觉：固定左端点，从长度 L 开始往右扩窗。比较扩一格前后的平均：

$$(a_1 + \dots + a_L) / L - (a_1 + \dots + a_{L+1}) / (L+1) = (a_1 + \dots + a_L - L * a_{L+1}) / (L(L+1))$$

当 a_{L+1} 足够大时该式 < 0 ，于是冒出念头：“扩窗后平均变小了，就别再扩了”，并据此想用双指针 / 单调扩窗。

卡点：这一步只是某个固定左端点、某一步扩窗的局部比较。平均值关于窗口长度既不单调也不单峰——这一格降下去，再扩两格完全可能反弹更高。剪枝会漏掉真正的最优。

反例（用 ground truth 打掉直觉）： $a = [10, -100, 10, 10, 10]$ ， $L = 2$ 。

从 $[10, -100]$ (avg -45) 扩到 $[10, -100, 10]$ (avg -26.7) 确实更低；但继续扩到 $[10, -100, 10, 10, 10]$ (avg -12) 又拉回来了。“ < 0 就停”会在第一步停掉，错过后面的反弹。

Key takeaway. Average vs. window length is non-monotone (and not unimodal). The two-pointer prune has no valid monotone invariant to ride on — this path is dead.

平均值关于窗长非单调、非单峰；双指针没有可依赖的单调量，此路不通。

Attempt 2 — 二分的猜想（注意：单调性此刻仍存疑）

既然平均本身不好贪心，转念：能不能二分答案？这只是一个猜想。二分要成立必须有单调的判定，而“对什么量单调”此刻还没想清楚——下面的弯路正是源于这里没钉死。

设判定 $J(x)$ ：“是否存在长度 $\geq L$ 的区间，其平均满足某种与 x 的关系”。关键问题：这个“关系”到底该是 $= x$ 还是 $\geq x$ ？先记下这个悬而未决的岔口。

Key takeaway. Binary search is only a hypothesis here. Its validity hinges on a monotone predicate that has NOT yet been pinned down.

二分此刻只是猜想；其成立与否取决于一个尚未确定的单调判定。

Attempt 3 — $g(k)$ + 哈希找相等（关键弯路）

直觉：把 $\text{avg} = x$ 代入。若区间 $(j, i]$ 平均恰为 x ，则 $P[i] - P[j] = x(i - j)$ ，移项得：

$$\text{令 } g(k) = P[k] - k \cdot x, \text{ 则 “区间 } (j, i] \text{ 平均} = x \text{”} \Leftrightarrow g(i) = g(j)$$

于是想：扫一遍，用 hashmap 存 g 值，若出现 $g(i) = g(j)$ 且 $i - j \geq L$ ，就说明 x 可命中。

这个命题本身没错——对“是否存在区间平均恰好等于 x ”，哈希找相等逻辑是对的。错在它对二分没用：

(1) 二分需要的判定是“平均 $\geq x$ ”，不是“ $= x$ ”；(2) “ $= x$ ”是测度零事件，浮点下几乎永不命中；(3) 更致命：“ $= x$ ”这个判定不具单调性（比 x 小的 x_1 和比 x 大的 x_2 都可能恰好被某区间命中），无法支撑二分。

换句话说：这一步解了一个二分用不上的子问题，所以接不回主线。

Key takeaway. Hashing for $g(i)=g(j)$ correctly decides "average EXACTLY equals x " — but equality is measure-zero and, crucially, NOT monotone in x . It cannot drive a binary search.

哈希找相等正确判定“平均恰好 $= x$ ”，但相等是测度零、且关于 x 不单调，撑不起二分。

单调性辩论 — 真正逼出结论的地方

卡在 Attempt 3 后，反复纠结的全是“到底对什么量单调”：

- “某对 (i, j) 凑出了 x ， x 变小时，是否一定存在另一对 (i', j') ？”——想显式构造下一对下标，构造不出来，推不动。
- “固定下标比较 g 跨 x 的变化”——只跟踪了固定 (i, j) 的单调，没覆盖“最优 (i, j) 随 x 漂移”。
- “judge 跑一遍能判某个具体 x 可行不可行，但 judge 这个过程本身不自带传递性”——判定器 != 单调性证明，单调性得另证。
- “不能用 exact 的 x 去设单调性，因为 $x_1 < x$ 与 $x_2 > x$ 都可能可行”——这句话精确刻画了“ $= x$ ”版为什么没单调性。

收口的一念：把判定换成“最优（最大）区间平均 $\geq x$ ”。记

$$f^* = \max_{i-j \geq L} \text{avg}(j, i), \text{ 则 } J(x) = [f^* \geq x]$$

f^* 是一个定值。 $J(x) = [f^* \geq x]$ 关于 x 显然单调： x 越小越容易满足，存在唯一分界点 f^* ，左侧全 yes、右侧全 no。单调性到这一刻才真正成立——它是经过弯路和多轮辩论后才收口的结论，不是 Attempt 2 一开始的既成事实。

Key takeaway. Monotonicity is recovered ONLY after switching the predicate to "optimal average $f^* \geq x$ ". This is the late-arriving payoff of the whole detour, not an assumption made up front.

把判定改成“最优平均 $f^* \geq x$ ”后单调性才成立；这是整条弯路最后才兑现的结论，而非一开始的假设。

收口 — $\geq$ 版判定的 $O(n)$ 实现

单调性既立，全部难度压到一件事：给定 x ， $O(n)$ 判定“存在长度 $\geq L$ 的区间，平均 $\geq x$ ”。从正确的不等号出发：

$$\text{avg}(j, i) \geq x \iff P[i] - P[j] \geq x(i - j) \iff P[i] - i \cdot x \geq P[j] - j \cdot x$$

仍记 $g(k) = P[k] - k \cdot x$ （注意：现在是 $\geq$ ，不是 Attempt 3 的 $=$ ）。判定变成：

$$\text{存在 } j \leq i - L \text{ 使 } g(i) \geq g(j) \iff g(i) \geq \min_{j \leq i-L} g(j)$$

关键精确化：需要的不是“下标 j 最小”，而是 $j \leq i-L$ 范围内 $g(j)$ 的最小值。只要 $g(i)$ 不小于这个滚动最小值，就存在合法的 j 。

实现（单次 judge $O(n)$ ）：从左到右扫 i （从 L 到 n ）；每步先把刚变合法的下标 $j = i-L$ 的 $g(j)$ 并入滚动最小值 m ，再比较 $g(i) \geq m$ 。存在即返回 true。

易错：必须“先并入 $g(i-L)$ 、再比较 $g(i)$ ”，顺序保证 j 始终落在 $\leq i-L$ 的合法窗口里；并反了会把非法的 j 也算进来。

2. 最终解法（干净版） / Final Algorithm

(1) 预处理前缀和 P 。(2) 在值域 $[\min a[i], \max a[i]]$ 上二分候选平均 x 。(3) 对每个 x 跑上面的 $O(n)$ judge。(4) judge 为真则下界右移，否则上界左移，直到区间宽度 $< \text{eps}$ 。

```
judge(x): m = +inf; for i = L..n: m = min(m, g(i-L)); if g(i) >= m: return true; return false
```

3. 复杂度 / Complexity

单次 judge $O(n)$ 。值域宽度 $V = \max a[i] - \min a[i]$ ，二分到精度 eps 需 $O(\log(V/\text{eps}))$ 轮。总复杂度：

$$O(n * \log((\max a[i] - \min a[i]) / \text{eps}))$$

相比 $O(n^2)$ 暴力枚举所有区间，对 $n = 1e5$ 是决定性的提升。（存在 $O(n)$ 的凸包 / 单调队列做法，但二分判定是面试现场最稳、最易讲清的路线。）

4. 易错点清单 / Pitfalls

#	陷阱 / Pitfall	修正 / Fix
1	把平均当成关于窗长单调，用扩窗剪枝	反例 [10,-100,10,10,10] 直接打掉；改二分
2	判定用“ $= x$ ”（哈希找相等）	“ $= x$ ”测度零且不单调；必须用“ $\geq x$ ”
3	把“可行性单调”当成一开始的前提	它要对“最优平均 $f^* \geq x$ ”才成立，是最后才收口的结论
4	“ j 最小”指下标最小	要的是 $g(j)$ 在 $j \leq i-L$ 范围内的最小值
5	先比较 $g(i)$ 再并入 $g(i-L)$	必须先并入再比较，保证 j 落在合法窗口
6	浮点用相等判定 / 精度设太松	用 $\geq$ 配 $\text{eps}=1e-6$ ，固定二分轮数（如 100 轮）

5. 元教训 / Meta-lessons（面试现场）

· ground truth 当裁判，不信断言。非单调这件事，靠一个具体反例 [10,-100,10,10,10] 一秒钉死，远胜口头争辩。

· 弯路本身有价值。“ $=$ ”

x ”那条哈希线虽然接不回二分，但正是它逼出了“到底对什么量单调”的追问，最终导向 $f^* \geq x$ 。把弯路记下来，复盘时才看得见单调性是怎么被逼出来的。

- claim-first 的反面教材。这条轨迹之所以长，部分因为单调性没在前面先立。Staff 级的讲法应是：先 30 秒给“二分答案 + 对最优平均单调”的高层框架，再落 $g(k)$ 与滚动最小值的机制 —— 而不是反过来从机制摸索框架。
- sparring 中判定方也会错。本轨迹里，“哈希判等”这个本身成立的命题一度被反复误判为错。教训对称：无论面试官还是 sparring partner，判定都可能出错，候选人要敢用反例 / ground truth 顶回去，并坚持“我问什么你答什么”的收敛纪律。